IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 20 June 2026, accepted 7 July 2026, date of publication 13 July 2026, date of current version 21 July 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3712592

RESEARCH ARTICLE

Explainable AI-Driven Metrics for Transparent Software Quality Prediction

ABDULAZIZ ATTAALLAH[1] AND KHALIL AL SULBI[2]
[1]Computer Science Department, Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia
[2]Department of Computers, Al-Qunfudah Engineering and Computing College, Umm Al-Qura University, Mecca 28815, Saudi Arabia

Corresponding author: Abdulaziz Attaallah (aattallah@kau.edu.sa)

This work was supported by the Deanship of Scientific Research (DSR) at King Abdulaziz University, Jeddah, under Grant GPIP: 2000-611-2024.

ABSTRACT Accurate software quality prediction is critical for early defect identification and effective allocation of testing resources. Although machine learning (ML) and deep learning (DL) models have significantly improved defect prediction performance, their opaque nature limits transparency, interpretability, and practitioner trust. This study presents an Explainable Artificial Intelligence (XAI) driven framework for developing composite, transparent software quality metrics that integrate predictive accuracy with multi-level interpretability. The framework has been implemented and empirically validated using multiple real-world datasets. The framework combines static code metrics, process metrics, and developer activity metrics to construct robust defect prediction models using Random Forest, Gradient Boosted Trees (e.g., via the XGBoost implementation), Support Vector Machines (SVM), Logistic Regression, and attention-based Neural Networks. To enhance transparency, model agnostic explanation techniques, including SHAP and LIME, are integrated with in-model attention mechanisms to provide global and local explanations of predictions. Experiments conducted on PROMISE, open-source, and industrial datasets demonstrate that the proposed composite explainable metric achieves superior performance on the PROMISE (NASA-JM1) dataset, with the Gradient Boosted Trees model reaching an Accuracy of 0.85 and an AUC of 0.88, and consistently outperforms traditional and opaque baselines across the Eclipse JDT, Apache Commons, and Industrial datasets. Quantitative faithfulness analysis shows strong alignment between SHAP explanations and model behavior (Pearson's r=0.81). A controlled user study involving software developers reports a 14% improvement in decision accuracy and significantly higher confidence levels ($p<0.01$) when using explainable metrics. The framework further incorporates interactive visual dashboards and textual summaries to support actionable decision-making during quality assurance processes. By bridging the gap between predictive performance and interpretability, this research demonstrates that explainability not only enhances trust but also improves the effectiveness of practical defect triage. The methodology provides a scalable and extensible foundation for transparent AI-driven software engineering tools.

INDEX TERMS Software engineering, XAI, defect prediction, machine learning, code quality, SHAP, LIME.

I. INTRODUCTION

Software quality estimation is important in software engineering to enable the identification of faults at early stages of the development process and improve the maintainability, reliability, and performance of software systems. It helps developers focus testing efforts where needed most and provides valuable insight into how best to allocate limited resources to meet tight deadlines while developing high-quality software products. For decades, traditional software quality metrics such as cyclomatic complexity, code churn, and defect density have been used to measure software quality and identify faulty modules in software systems [1], [2]. More recently, many Artificial Intelligence (AI) driven predictive models have been developed to estimate the likelihood of a module being faulty by analyzing historical data

The associate editor coordinating the review of this manuscript and approving it for publication was Sotirios Goudos.

and various source-code characteristics [3]. The predictive performance of these models far exceeds the performance of traditional threshold-based approaches. Despite significant advancements in the field, one of the biggest challenges is the transparency and interpretability of making predictions. Many AI based models developed for quality estimation purposes operate as ''opaques,'' predicting potential issues in modules without explaining the defects [4]. The lack of transparency hinders developers' adoption of predictive quality estimation tools. This, in turn, limits their effectiveness when applied to real-world software development and testing processes [5]. If a team cannot see the reasons, they will struggle to identify ways to correct problems for refactoring or testing.

One of the main motivations for integrating XAI into software quality estimation is to create a new generation of transparent, interpretable, and actionable quality metrics that produce more than just predictions. They provide context around those predictions and highlight the features that contributed to them [6]. The application of XAI techniques to software quality metrics will enable developers and testers to gain contextual understanding of the predictors of defects in a software system. This will promote trust and encourage more informed decision-making during the quality assurance phase of the software development lifecycle. Research conducted over the last few years has demonstrated the utility of XAI tools such as LIME (Local Interpretable Model-agnostic Explanations) and SHAP (SHapley Additive exPlanations) to several software engineering tasks [7]. However, there has been little exploration of a systematic approach to creating explainable quality metrics specifically designed for predictive software quality estimation.

This paper presents a novel framework that uses XAI techniques to develop quality metrics that provide accurate predictions of software quality and understandable explanations. We use real world datasets to build AI-based predictive models and incorporate explanation-generation techniques. The contributions of this paper include (i) the development of explainable quality metrics based on AI interpretation outputs, (ii) an implemented framework illustrating explainability in fault prediction, (iii) empirical validation demonstrating accuracy of predictions and practical usefulness of generated explanations, and (iv) a controlled user study demonstrating a 14% improvement in developer decision accuracy and confidence during defect triage. This paper bridges the gap between high-performing AI models and their practical applications in quality estimation by providing greater transparency and trust.

The rest of this paper is structured as follows: Section II describes a systematic literature review related to AI-based quality estimation and explainability in software engineering. Section III proposed the methodology. Section IV detailed about the implementation. Section V explains the experimental assessments and results of the explainable quality metrics framework. Section VI addresses the threats to validity, section VII compared the existing of the proposed methodology. Section VIII discuss limitations, and assumptions of this research. Section IX concludes the paper and outlines further research.

II. LITERATURE REVIEW

A. REVIEW GOALS AND RESEARCH QUESTIONS

The goal of the review is to assess the current state of research on software quality estimation using AI models and XAI techniques. This literature review will investigate and evaluate the most recent methods, tools, and metrics for measuring quality, and identify challenges for further research related to transparency and interpretability in predicting software quality. The review was designed to assess how AI based automated quality assurance processes are being adopted in industry, while recognizing the need to give explanations to increase users' confidence and enable practical use.

To guide the execution of this review process, five research questions (RQs) were developed:

- RQ1: Which AI and Machine Learning techniques are most often utilized for estimating software quality?
- RQ2: How are existing Quality Metrics defined and utilized for predicting defects and assessing software quality?
- RQ3: Which Explainability techniques have been utilized in Software Engineering for estimating quality or conducting testing?
- RQ4: What are the main Challenges/ Limitations/ Open Problems that researchers have identified in terms of Explainability of software quality prediction in the literature?
- RQ5: Where are the Gaps that support the necessity of developing XAI-based quality metrics?

B. SEARCH STRATEGY

To summarize the existing research in the area, a review was conducted across academic databases, including ACM Digital Library, ScienceDirect, SpringerLink, IEEE Xplore, and Google Scholar. We selected foundational research, as well as the most current and anticipated research on software quality estimation, AI-based prediction, and XAI from 2010 to 2026. The study captures trends, identifies emerging methods, and highlights advancing explainability techniques being developed and applied to software engineering estimation. This reflects a transforming research trajectory, projected to continue through 2026. Search terms and combinations of keywords include ''Software Quality Estimation,'' ''Defect Prediction,'' ''Software Metrics,'' ''Machine Learning,'' ''Artificial Intelligence,'' ''Explainable AI,'' ''XAI,'' ''Software Testing,'' ''Model Interpretability,'' and ''Quality Metrics Explainability.''

To ensure the selection of relevant papers, we included Peer-reviewed journal articles and conference papers published or accepted between 2010 and 2026. Studies that used machine learning or AI based techniques for software quality prediction or defect detection were included. Papers

that addressed explainability or interpretability aspects of software engineering models and predictions were selected. Articles that proposed new quality metrics, explanation methodologies, or empirical validations through experiments or user studies were also included in the review process. The exclusion criteria include non-English language publications, publications that did not relate to software quality estimation or explainability in software engineering, and editorial articles, very short papers, and workshops with no full paper available or duplicate publications.

C. SUMMARY OF STUDIED PAPERS

We observed the strict search screening criteria and identified seventy-eight (78) papers on the basis of the title and abstract. After full-text review, only twenty-seven (27) papers were ultimately deemed relevant and selected for analysis. These 27 papers reviewed fell into three general thematic categories and are outlined in Table 1:

TABLE 1. Categorization of reviewed literature.

Category	No of Papers
Quality Estimation Using Artificial Intelligence	8 [1, 3, 8-11, 20-21]
Explainability in Software Engineering	11 [4-7, 12-13, 22-26]
Design and Use of Quality Metrics	8 [2, 14-18, 27-28]

Quality estimation using AI: Several studies utilize machine learning techniques, including random forests, support vector machines, and neural networks, to identify potential faults in software modules [8], [9]. Many of the machine learning techniques described in these studies primarily use static code metric features, process data, and developer behavior log features [10]. Some studies include additional feature types. Lessmann et al. compared classifier performance and found that ensemble approaches can achieve defect prediction accuracy above 80% when combined [3]. Nagappan et al. showed that predictor variables such as code churn and historical fault records can significantly improve the ability to predict which modules are most likely to contain defects [1]. More recently, systematic evaluations of fault prediction performance have demonstrated that ensemble and benchmark classification approaches continue to advance the state of the art in software defect prediction [20], [21].

Explainability in Software Engineering: There is an increasing body of research focused on adding explainability to AI models used for software analytics and defect prediction [4], [5]. Research on explainability has focused on developing tools that help explain how an AI model arrived at its prediction [4]. An example of one of these tools is LIME, which provides local interpretations of opaque models [4]. Another tool that has been utilized to provide explanations of how an AI model made its predictions is SHAP (SHapley Additive exPlanations). This tool provides a method for identifying which features had the greatest influence on the AI model's prediction [6]. Recent studies have added attention mechanisms to enhance the interpretability of these AI models [22]. Studies have also shown that providing explanations of how an AI model arrived at its predictions can improve developers' trust in the model [12]. Studies have also demonstrated the use of causal and counterfactual explanation techniques to provide actionable information to developers regarding what actions they can take to prevent future defects [23], [24]. Some of the more recent studies focus on cross-project defect prediction approaches that leverage domain knowledge to improve generalizability across diverse software development environments [25], [26].

Design and use of Quality Metrics: The traditional quality metrics, such as cyclomatic complexity, number of lines of code, and defect density, are still commonly used today in quality evaluations [2]. Researchers have developed new composite metrics that combine static code metrics and dynamic process metrics to increase the reliability of quality predictions [14]. While these composite metrics may increase the reliability of quality predictions, they are generally opaque and lack interpretability. They fail to communicate to developers how these metrics will affect the quality of their product [15]. Recent research advocates the use of XAI to provide transparency into traditional quality metrics [27], [28]. However, this integration is still very limited and developing.

D. FINDINGS FROM THE REVIEW: CURRENT MODELS, CHALLENGES, AND LIMITATIONS

Although substantial work has been reported in the literature on using AI to improve the accuracy of predictive techniques for software quality, there is a significant lack of transparency about its use. The key observations from the literature review are summarized as follows:

- High prediction accuracy vs. low interpretability: Deep Neural Networks and Large Ensemble methods, which achieve high accuracy in predicting software quality, are difficult for developers and testers to understand, let alone use [3], [8], [20].
- Quality Metrics and Explanation Research are Decoupled: It has been inferred that most of the Quality Metrics in use are traditional, and the explanation techniques are decoupled from quality estimation frameworks [14], [16], [27].
- Human-Centric Evaluation: Only a few studies have included development team feedback to evaluate the clarity, trustworthiness, and usability of explanations provided through user studies or field experiments [12], [13], [22].
- Constraints of Data Heterogeneity and Quality: Datasets that contain imbalances of data, differences in project characteristics, and subjective labeling can make it difficult to create explanations that are reliable and consistent [9], [10], [21].

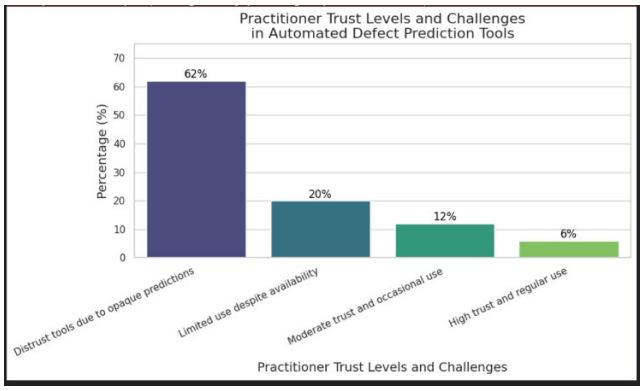

FIGURE 1. Practitioner trust levels and challenges in automated defect prediction.

- Domain Variability Adds Complexity: The diversity in size of software projects, domain, programming languages used, and software development processes makes it challenging to develop a universal XAI framework for estimating software quality [17], [25].

As shown in Figure 1, a majority (62%) of surveyed software development practitioners expressed distrust in using automated defect prediction systems due to their lack of transparency in explaining why predictions were made [26].

E. IDENTIFICATION OF RESEARCH GAPS

The findings of our systematic review indicate that there are a number of major limitations in the present research base. This supports the need to develop an integrated set of explainable quality metrics tailored to predicting software quality.

To begin, many researchers continue to formulate their metrics using narrow, static, or process-based approaches. Few researchers incorporate direct methods of explaining the metrics they develop. Further, most researchers do not create composite, actionable quality metrics that developers can easily understand and utilize. As such, many prior works were limited to either global feature importance or local interpretation at the instance level [2], [14], [27].

In terms of explanation granularity, we find that this dimension was also limited. Most prior work provides either global feature importance or local (instance-level) interpretations. However, very few prior works provided both types of explanation as part of a single, multi-level approach that could be used to address varying stakeholder requirements and context [4], [6], [22].

Lastly, user-validated evaluations and human centered assessments were rare. Prior works did not conduct robust experiments or field-studies with practicing developers and testers to examine whether explanations affected decision-making accuracy, decision-making confidence/trust and usability in realistic workflow scenarios [12], [22], [25]. Such a lack of practitioner input severely limits the real-world relevance of many prior approaches.

Finally, in terms of deployment-readiness and industrial applicability, most of the works cited in the literature have been restricted to assessing model performance on academic and small open-source data sets. Existing literature places limited emphasis on scaling these models to large industrial projects or on integrating them into real-world development pipelines. It also falls short of creating interactive visualization tools to facilitate action-oriented software quality management practices [30], [31].

These clearly identifiable voids serve as the motivation for creating a unified, scalable framework that combines predictive accuracy, multi-level explanation capabilities, user-centered assessment, and practical application across different software development domains. Our developed framework closes this gap through empirical validation across multiple data sources and through user studies with practitioners from varied development domains.

III. PROPOSED METHODOLOGY

In this section, we describe an integrated approach combining predictive modeling and XAI to enhance the state of the art in software quality estimation. A detailed description of the research process from data collection to the deployment of machine learning models to the creation of new XAI quality metrics, giving transparency and accuracy in predicting quality, is presented. Unique to our proposed method is a multi-level mechanism for explaining predictions, applying model-agnostic approaches such as LIME and SHAP to interpretable architectures that employ attention mechanisms. The approach presented in the paper provides actionable intelligence to developers and other stakeholders, enabling informed decisions about software quality management. It collects user feedback to improve explanations and model trustworthiness. Our framework provides a unique methodological roadmap for predicting software defects while empowering users to develop high-quality software. In summary, the methodology integrates predictive modeling and XAI into a single framework that enables effective prediction of software defects. It further empowers developers and other stakeholders to use high-quality information to make informed decisions in software quality management.

A. OVERALL RESEARCH DESIGN

The methodology employed is a sequential process comprising the design, development, and validation of an XAI-based predictive quality metric for software projects. In addition to being sequential, the process illustrated in Figure 2 includes a human feedback loop. It consists of collecting and preprocessing data, training a model, integrating explanations into the model, and evaluating both the model and the explanations. The developed XAI driven quality metric framework is illustrated in Figure 3 as an extension of this research flow, which details the primary elements of the Software Quality Prediction Process. The framework uses multiple layers of explainability to illustrate the stages of data collection and feature extraction for AI-based prediction, multiple levels of

explanation, composite metric creation, and iterative refinement using human feedback. It demonstrates the continuous and interactive nature of the process, combining AI model output, explanation processes, and user input into a comprehensive, transparent quality measurement system.

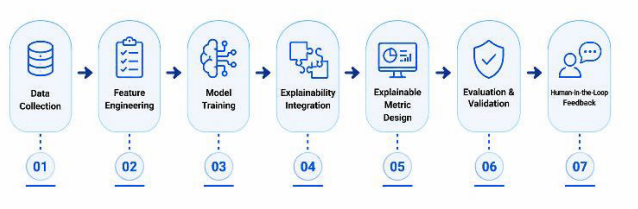

FIGURE 2. Research workflow from data collection to human feedback.

The methodology ensures iterative refinement through human feedback, pragmatically enhancing explanations and metrics.

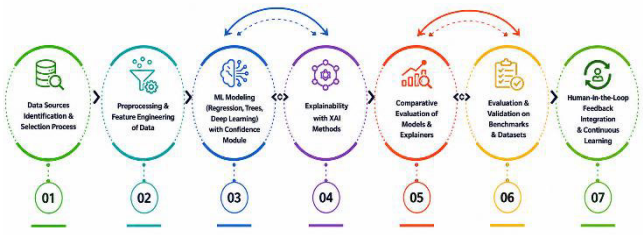

FIGURE 3. Explainable AI-driven software quality metric framework and interactions.

B. DATA SOURCES

Our research is based on numerous authentic, highly regarded, and widely recognized datasets that depict an extensive range of realistic software defect prediction scenarios. These datasets will provide a solid basis for training models and conducting empirical analysis. The datasets are defined below [29], [30]:

- JM1 project dataset in the NASA PROMISE repository (with over 10,000 modules and 20 + metrics). Defect-labeling data was documented by developers from 2000 through 2005.
- Eclipse JDT open source dataset (over 8,000 modules), consisting of a wide variety of static, process, and developer activity metrics. Data was collected from 2001 through 2007.
- Apache commons open source dataset (over 12,000 modules) with 25+ features and labels documenting defects. Data was collected between 2005 and 2015.
- Proprietary industrial dataset (anonymously provided from proprietary software projects), with approximately 15,000 modules and 40+ features. Features reflect large scale commercial development processes from 2015 to 2022.

Datasets were evaluated to ascertain their level of realism, diversity of software domains and overall representation of code, process and developer-related features. Therefore, they can serve as a basis for strong generalizability and representative evaluation of the proposed AI-driven predictive framework. Table 2 summarizes key datasets considered.

TABLE 2. Summary of software quality datasets.

Dataset	Source/ project	Modules	Features	Labels	Year Range
NASA-JM1	NASA PROMISE	10000+	20+	Defect/No	2000-2005
Eclipse JDT	Open Source	8000+	30+	Defect/No	2001-2007
Apache Commons	Open Source	12000+	25+	Defect/No	2005-2015
Industry Dataset	Proprietary (anon)	15000 (approx.)	40+	Defect/No	2015-2022

C. FEATURE SELECTION AND ENGINEERING

Feature selection has a significant impact on the accuracy and interpretability of predictive models and, therefore, on explainable quality metrics. The features affecting software quality and defect proneness are numerous and are captured using a combination of static code metrics, process metrics, and developer activity metrics. In this way, our multidimensional approach captures the code's structural complexity, its relationship to faults, and its evolution over time, and developers' interactions with it.

- Static Code Metrics, including Cyclomatic Complexity, LOC, Depth of Inheritance Tree, and Coupling Metrics, represent intrinsic measures of code complexity and maintainability [2], [29] and have been established as good indicators of potential fault-proneness through years of empirical research.
- Process Metrics, including Code Churn, Number of Commits, and File Age, reflect the dynamic nature of software development and indicate the historical context and temporal relationships between code modifications and fault-proneness [30], [31].
- Developer Activity Metrics, including the number of Developers touching a Module and the history of Defect Fixes, provide additional information regarding the interaction between developers and code. Research has demonstrated that modules with more developers touching them and developers who have made more complex prior fixes tend to be more fault-prone.

Therefore, this combination provides a balance among code structure, historical evolution, and developer behavior, enhancing the model's ability to generalize across different projects and context scenarios. A limitation of the developed approach was the availability and quality of process and developer data, which could vary significantly across projects and between open-source and industrial settings, thereby affecting the robustness of the model.

Standardization was applied to all selected features to allow comparison on a common scale, and multicollinearity was examined to avoid redundancy and overfitting. Although Dimensionality Reduction Methods, such as PCA, can improve efficiency, caution was exercised because they may decrease feature interpretability, which is critical for explainable models. Therefore, PCA was only used to reduce dimensionality of the feature space if it preserved or could be remapped to an interpretable feature space.

D. FEATURE IMPORTANCE ESTIMATION

Tree-based Models (Random Forest and XGBoost) have been initially employed to estimate feature importance for the selected feature set using their internal mechanisms for calculating feature contributions. They were selected for their ability to produce feature contribution scores that support transparent analysis and are widely accepted as explanations of the features' contributions to the model's prediction [32], [33]. They possess robustness to noisy or correlated features and strong predictive performance, making them suitable baselines. Figure 4 illustrates the feature-importance scores obtained from a Random Forest model trained on the empirical software quality datasets described in Section III-B (PROMISE NASA-JM1, Eclipse JDT, Apache Commons, and the Industrial dataset, harmonised to a common feature schema as detailed in Section IV-B). The figure is provided here, in the methodology section, as a reference exemplar that illustrates how static code, process, and developer-activity features contribute to defect-prone modules; the corresponding per-dataset experimental results that confirm this ordering are reported in Section V (Tables 5- Table 8). Actual experiments with resulting visualizations have been presented after training and evaluating a real dataset.

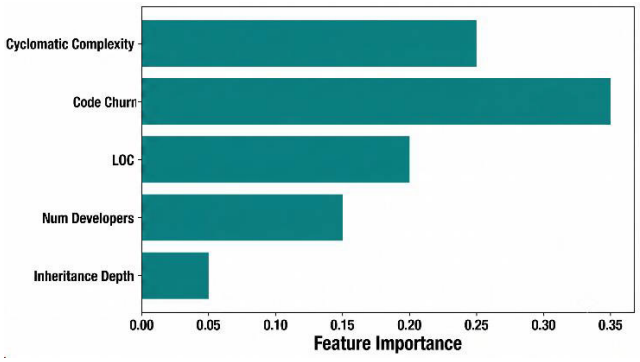

FIGURE 4. Feature-importance scores obtained from the Random Forest model trained on the empirical software quality datasets (PROMISE NASA-JM1, Eclipse JDT, Apache Commons, and the Industrial dataset), highlighting the respective impact of static-code, process, and developer-activity feature categories.

In this work, it was assumed that feature importance calculated from these models accurately reflects each feature's causal relevance, although this assumption has limitations. Therefore, the feature importance has been cross-validated with domain expertise, and additional explanation has been provided through post-hoc explanation techniques, such as SHAP.

E. AI/ML MODELS FOR QUALITY PREDICTION

The use of a variety of Machine Learning (ML) algorithms is required to obtain a comprehensive model of software quality, especially when attempting to capture the complexity of relationships between attributes and defect-prone software. This study utilized a selection of AI/ML models that balance predictive performance, interpretability, and computational cost.

- Random Forests (RF) were chosen because they have several desirable properties. They are robust to overfitting, can effectively process high-dimensional, correlated features, and produce interpretable feature importance metrics [34]. The aggregating decision-making approach used in RF provides greater generalization and stability in predictions than single decision trees. The transparent structure of RF models lends itself well to generating explainable quality metrics and allows for the simple identification of influential software metrics.
- Gradient Boosted Trees (e.g., via the XGBoost implementation) was chosen because it has been demonstrated to be one of the highest-performing AI/ML algorithms and is scalable to large datasets [35]. This model delivers high predictive performance and supports advanced interpretability techniques, such as SHAP (SHapley Additive exPlanations) [36]. The SHAP tool generates local and consistent explanations of how each feature contributes to the predicted outcome and aligns perfectly with the goal of developing transparent, actionable quality predictions. XGBoost models can be optimized by tuning many hyperparameters, but care must be taken to prevent overfitting.
- Neural Networks (NNs), along with Neural Network architectures utilizing attention mechanisms, were employed to evaluate potential nonlinear and high-complexity patterns in software data [37]. Attention allows NNs to selectively focus on critical input features, thereby enabling inherent interpretability of the network's output by identifying which input features contribute most to the prediction. NNs typically require larger training sets and more computing resources to train. While attention improves the interpretability of NNs, it does not achieve the level of transparency offered by tree-based models. Therefore, incorporating NNs into this research offered an opportunity to compare traditional AI/ML algorithms with more advanced deep learning methods within the same predictive framework.

In order to provide reliable baseline models and assess the benefits of employing complex models, simpler yet historically important algorithms such as Support Vector Machines (SVM) and Logistic Regression (LR) were utilized [38]. SVM and LR provide ease of implementation and

interpretability, making them suitable for comparison purposes. The inclusion of these models ensured that any improvements in performance and interpretability that are realized through the employment of more complex models are demonstratively justified.

We performed systematic hyperparameter tuning to obtain robust, replicable results from our models. We used both grid search and Bayesian Optimization (using GridSearchCV and Optuna) to determine the best hyperparameter combinations within stratified k-fold cross-validation on the training set. Each model had its own unique Hyperparameter Search Space that is defined below:

Model: Random Forest

- Number of trees:100, 200, 300
- Maximum tree depth: None, 10, 20, 30
- Minimum samples split: 2, 5, 10

Model: Gradient Boosted Trees (e.g., via the XGBoost implementation)

- Number of estimators: 100, 200, 300
- Learning rate: 0.01, 0.1, 0.2
- Maximum tree depth: 3, 6, 9
- Subsample ratio: 0.5, 0.75, 1.0

Model: Neural Network

- Number of layers: 2, 3, 4
- Hidden units per layer: 64, 128, 256
- Dropout rate: 0.1, 0.3, 0.5
- Learning rate: 0.0005, 0.001

Model: Support Vector Machine (SVM)

- Kernel type: linear, rbf
- Regularization parameter: 0.1, 1, 10

Model: Logistic Regression

- Regularization type (penalty): l1, l2
- Regularization strength: 0.01, 0.1, 1, 10

The Optimal Hyperparameters were determined by averaging the F1-Score across all the CV Folds. This extensive tuning process helped us achieve an even balance between the performance metrics while minimizing the risk of over-fitting.

Emphasis has been placed on achieving balanced performance across metrics such as accuracy, F1-score, precision, recall, and AUC (Area Under Curve) in order to address the issue of class imbalance in defect datasets and the differing relative importance of false positives and false negatives in quality assurance [43].

While the chosen machine learning algorithms were all good choices that offer a balance between high predictive power and high interpretability, they each come with important assumptions and limitations. The tree-based methods offer many advantages, such as feature importance; however, they cannot fully capture complex causal interactions in software metrics and may therefore oversimplify these relationships. Although attention-based neural networks have proven very effective at detecting quality issues in software by focusing on relevant information, their need for substantial amounts of labeled data can limit their effectiveness in small projects. Use of transfer learning or data augmentation may help overcome this limitation. However, neither was in the scope of this research. The simpler models, such as SVM and Logistic Regression are highly interpretable. They may not be able to detect the non-linear dependencies that exist in many software quality-related features, and as such, would be less than optimal when used as a standalone quality prediction model. The feature sets available and quality can be quite different from those found in an industrial setting, and as such, can limit the robustness and generalization of the developed models. Hyperparameter tuning and model evaluation was performed under the assumption that the dataset is representative of the problem space and has sufficient size and complexity to support the chosen metrics (accuracy, F1-Score, AUC). The scope of the paper was limited to defect prediction in the software quality domain based on static, process, and developer metrics, it is important to note that many other runtime and environment specific factors can also have significant effects on software quality in practice.

F. XAI TECHNIQUES INTEGRATION

To give transparency and actionability to predictive decisions on software quality, an explanatory approach at two complementary levels, namely post-hoc, model-agnostic explanations and an in-model attention mechanism, has been applied.

Post-hoc Model-agnostic Explanations: Leveraging the best-in-class explanation frameworks such as LIME [39], and SHAP [40] to produce local (at the instance level) and global (model-wide) explanations of predictive output. LIME uses a simple, interpretable model to approximate the behavior of a complex model at a specific point of interest, identifying relevant features that affect the predictive decision. SHAP employs cooperative game theory to consistently attribute importance scores to features, producing a robust global interpretation of the model's behavior.

The rationale for choosing these techniques was their model-agnostic nature, which means the framework can produce explanations for predictive decisions made by any opaque model (e.g., Random Forest, Gradient Boosted Trees via the XGBoost implementation, and Neural Network). The post-hoc approaches alleviated the problem of interpreting complex models without reducing predictive accuracy. There were limitations associated with post-hoc approaches, such as potential instability in explanations due to sampling variability (especially LIME), and high computational overhead associated with applying the framework to large datasets or models.

In model attention mechanisms: Neural network architectures utilize attention layers to build intrinsic interpretability by identifying the most significant input features or regions involved in predictive decisions [41], [42]. With supporting accurate predictions, attention layers build interpretability into the model, as opposed to utilizing post-hoc analysis. The attention mechanism dynamically assigned weights to input features, thereby focusing the model's ''attention'' on

the most significant features of the feature space, which can be visually represented as heatmaps.

The application of attention mechanisms to sequence-based data or features with structured relationships, such as time-sequenced code changes or developer activity logs, was particularly effective. The use of attention mechanisms to provide interpretability is subject to research debate and does not necessarily correspond to causal feature importance. This represents a limitation of attention-based interpretability. Figure 5 depicts a SHAP summary plot produced by an Gradient Boosted Trees (e.g., via the XGBoost implementation) classifier trained on software quality metric data. In addition to illustrating global feature importance distributions and their effects on the models' predictions across individual test cases, this figure provides further insight into which quality factor(s) have the greatest influence on predicting when a software defect is likely.

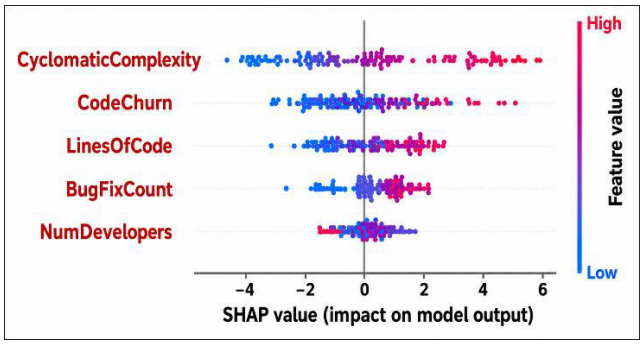

FIGURE 5. Illustrative SHAP summary plot for feature importance.

G. DESIGN OF EXPLAINABLE QUALITY METRICS

The primary contribution of this work was an integrated method for developing composite explainable quality metrics from predictive model output, enabling comprehensive and transparent evaluation of software quality. Composite explainable quality metrics differ from traditional metrics, which have been developed to give non-transparent risk scores. Composite explainable quality metrics combine AI Model Output (predictive model outputs) and Explainability Outputs (quantitative interpretations), including feature importance scores using SHAP and LIME, and attention weights. This combination allowed for a clear understanding of the predictive models, providing actionable information for development teams and project managers. For example, instead of identifying a module as high risk, the composite metric can identify why the module is high risk, such as high code churn, frequent commit history, and multiple developers involved [44]. The composite explainable quality metrics has been shown on an interactive visualization dashboard so that they may be effectively used by stakeholders for making decisions and interpreting results. These visualization dashboards include:

- Risk scores for each component/module.
- Rankings and heatmaps of contributing factors along with feature importance/attention weights.
- Temporal graphs displaying how changes in quality predictives result in changes in quality scores over time.

The above visualizations assisted stakeholders in determining which module is at risk. Much more importantly, they determined why that module is at risk. Determining why a module is at risk significantly helped in planning where to apply testing or refactoring and other quality assurance efforts. Figure 6 displays a prototype of what an interactive dashboard could look like. The dashboard includes composite explainable quality metrics in the form of a defect risk score, as well as the contributing features, displayed through SHAP waterfall and force plots that correctly represent the bidirectional (positive and negative) feature attributions returned by the additive explanation model.

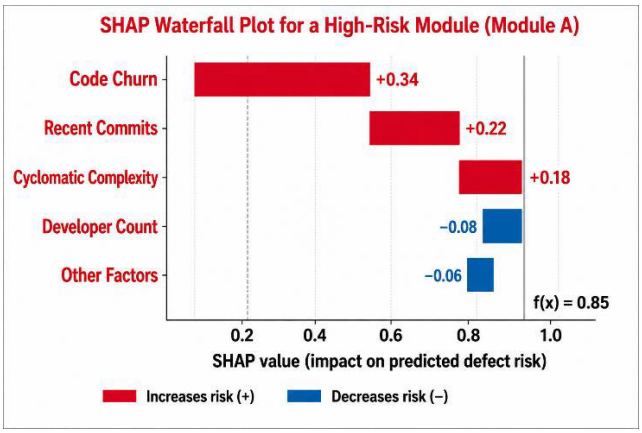

FIGURE 6. SHAP waterfall plot for a high-risk software module (Module A), decomposing the predicted defect risk into signed, per-feature contributions starting from the base value E[f(x)]. Positive (red) contributions push the risk above the baseline, while negative (blue) contributions pull it back toward the baseline; the cumulative sum yields the final predicted risk f(x). This visualization preserves the bidirectionality of SHAP values and replaces the previously used stacked bar representation.

Designing quality metrics with explainability helped address the problem that high-performing models are either untrusted by users or not used because they do not give transparent explanations (as in, opaque models) [45], [46]. Adding explainability to the design of quality metrics increased transparency in the process and makes it easier to hold people accountable for how the model works. This provided an opportunity for more granular, understandable feedback to users/practitioners who use the model to predict defects. This approach relied on high-quality output from the explainability mechanisms, and the quality of those mechanisms depends on both the AI models that generated the data being explained and the methodology used to produce the explanation. The complexity of the composite metrics created cognitive load for users, requiring careful UI/UX design in dashboards to maintain clarity. These metrics were designed to predict defects. If they are to be extended to include other quality attributes, such as performance or security, the feature sets

and the methods for achieving interpretability may need to be modified.

H. FORMAL DEFINITION OF COMPOSITE EXPLAINABLE QUALITY METRIC

To formally define the composite explainable quality metric, a software module described by a feature vector ($F = \{f_1, f_2, \ldots, f_n\}$) can be used. In addition, this module had a predictive model outputting a defect risk score ($S \in [0, 1]$) describing the estimated probability that the module has defects.

Additionally, explainability techniques like SHAP generated a collection of feature attribution scores ($\Phi = \{\varphi_1, \varphi_2, \ldots, \varphi_n\}$) that each quantifies the effect of one feature (f_i) on the calculated risk score (S). Normalized attention weights for models using attention are represented as ($A = (a_1, a_2, \ldots, a_n)$), which describe how much emphasis each feature receives when determining its predicted risk score.

Therefore, the composite explainable quality metric (Q) combines all three to create a unified quality metric from the combination of prediction performance and interpretability by equation 1:

$$Q = \alpha.S + \beta. \sum_{i-1}^{n} \omega_i.g(\varphi_i, a_i) \quad (1)$$

where:

- $\alpha, \beta \in [0, 1]$ are coefficients that control how much the prediction component contributes to the final Q value versus the explanation component (where $\alpha + \beta = 1$).
- ω_i represents the normalized importance of each feature in the model.
- $g(\varphi_i, a_i)$ represents some form of aggregation of the φ_i (feature attribution) and a_i (attention) scores for the i^{th} feature; in practice either a weighted sum or product depending upon available information.

This allows Q to include both the actual risk associated with predicting defects and the proportionate contribution of each feature to that risk. The ability to combine multiple aspects of performance into a single metric creates greater transparency in understanding the relationship between risk and specific features.

In practice, interactive visualization dashboards represent (Q) alongside decomposed contributions $\{\omega_i g(\varphi_i,ai)\}$, empowering stakeholders to prioritize testing and refactoring activities effectively.

I. EXPLANATION PRESENTATION (VISUAL AND TEXTUAL)

Our explanation presentation framework has been designed to incorporate both visual and textual aspects, in line with HCI and Cognitive Psychology. This assists users in understanding, trusting, and making decisions about their software quality prediction tasks [47], [48].

Design rationale: A successful presentation of an explanation should minimize the cognitive effort required to process it while delivering defined, actionable information to stakeholders, including developers, testers, and project managers.Bar charts of mean absolute SHAP values are used to summarise the relative importance of each feature contributing to a particular Defect Risk Score, while SHAP waterfall and force plots are used at the instance level so that the bidirectional (positive and negative) feature attributions produced by the additive SHAP model are rendered without distortion [47]. These graphical displays allow users to identify patterns and rank the most influential metric(s) to test or repair.

The use of Heat Maps derived from Attention Weights in Neural Network Models enables users to spatially identify which features, or groups of features, the model relied on when making its predictions. This display can assist developers in identifying input areas that require further review or action.

Natural Language Generation (NLG) techniques are also employed to generate concise, human-readable textual summaries that convert complex model outputs and feature attributions into narratives understandable to non-technical stakeholders and serve as audit trails for the model's predictive reasoning [48].

Explanation presentation model: At a conceptual level, our explanation presentation model will map raw explainability output (i.e. Predictive Scores, Feature Attribution, Attention Weights) into forms of representation that are cognitively accessible and support user work flows through the following flow:

Explainability Output → Visual/Textual Encoding → User Perception & Comprehension → Informed Decision-Making

Explainability Output: Quantitative data defining feature influence and model focus [7], [40].

Visual / Textual Encoding: Displaying this data through bar charts, heat maps, and textual summaries [47], [48].

User perception and comprehension: Users comprehend this information through cognitive heuristics, facilitated by a clear presentation.

Informed decision making: Users leverage these insights to drive defect triage, test prioritization, and code maintenance.

This mapping follows established explanation frameworks in the XAI literatures [4] and [7], focusing on translating the model's internal workings into usable knowledge for practitioners.

Mapping explanation outputs to user decisions: The visual dashboards developed for this research incorporated the following types of visualizations:

- Bar charts of mean absolute SHAP values, together with SHAP waterfall and force plots for individual predictions: to represent the relative importance of individual features and the decomposition of quality metrics into their constituent parts.
- Heatmaps: To show the weighted attention of features within a neural network, which can be used to identify key inputs that have the most influence on a model's prediction.

- Summary overviews: To provide users with an at-a-glance view of the overall risk score of each module with some explanatory notes regarding the reasons behind those risk scores.

We chose each visualization component based on the cognitive tasks necessary for effective quality-assurance decision-making: ranking, pattern identification, and narrative comprehension.

Iterative user centric refining: Both the visualization-based and textual-explanation designs have undergone iterative refinement based on structured feedback gathered from professional developer and tester participants engaged in user studies. Iterative refinement ensures that all elements of the system meet user requirements, improve clarity and trustworthiness of the explainable metrics within the real-world workflows practiced by software developers and maintainers.

Supporting visualization: The SHAP Summary plot in Figure 7 was generated using the gradient boosted trees (XGBoost implementation) model trained on the empirical software quality metrics described in Sections III-B and IV-B (PROMISE NASA-JM1 dataset) to predict defect risk. The plot is therefore derived from the same experimental data used to produce the predictive performance reported in Tables 5 to Table 8, and the global feature-importance ordering shown is consistent with the SHAP values obtained from those experiments. This figure illustrates the type of visual explanation that is provided within our dashboard tool to help users understand how the models deliver interpretable results.

Integration of technical grounding and cognitive optimization in our comprehensive multimodal explanation presentation framework improved the confidence and informed decision-making in software quality assurance processes.

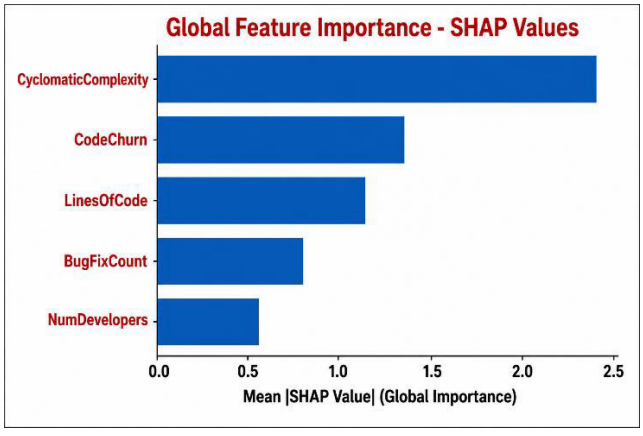

FIGURE 7. SHAP summary plot illustrating global feature importance and impact on model predictions for software quality estimation.

J. HUMAN-IN-THE-LOOP FEEDBACK INCORPORATION

The developed methodology incorporates an extensive human-in-the-loop feedback mechanism to ensure that the explainable composite quality metrics and their corresponding explanations support the activities of software development professionals.

Feedback Variables: During planned usability studies, involving professional software developers/testers and students, we obtain the following quantitative/qualitative feedback on various variables:

Clarity: Participants' subjective ratings concerning whether the provided explanations were easy to understand and clearly expressed.

Trust: Subjective assessments by participants of their self-confidence in the accuracy of the predicted quality scores/explanation outputs generated by the framework.

Decision accuracy: An objective evaluation of participants' performance on defect triage or quality-related decision-making tasks.

Usability: Qualitative feedback about the user experience, including the dashboard's layout, visualizations employed, etc.

Annotation protocol: Users participate in controlled defect triage tasks using interactive dashboards that display the composite explainable quality metrics, along with comprehensive visual/textual explanations. The feedback obtained via the surveys is standard across all participants. A likert scale is used to evaluate the participants' perceptions of clarity/trust/usability. Each user's choices are recorded to assess their correctness and consistency. Users will be asked to provide qualitative comments about the tool to elicit more nuanced perceptions of its usability.

All feedback is anonymized and securely stored to enable us to analyze it systematically.

Refining criteria: We use quantitative/qualitative analyses of the feedback to inform incremental improvements in the system. More specifically, we will refine the system when any one of the following conditions is met:

- There are statistically significant poor ratings in either clarity or trust, suggesting that we should revise explanation templates to be more easily understood by users or modify existing visual encoding schemes.
- There are recurring patterns of incorrect decisions that suggest we may want to adjust the thresholds for computing composite quality metrics or re-evaluate which features are being attributed to specific issues.
- There are proposed interface modifications that users repeatedly mention, which would require us to conduct a thematic analysis of these qualitative comments.

Integrating human input: Using these analyses, we make targeted revisions such as:

- Modifying the templates used to create textual explanations to make them clearer, less ambiguous, and more relevant to users.
- Automatically adjusting risk score thresholds in the computation of composite quality metrics so we find a better trade-off between false positives and false negatives, depending upon user preferences.

- Including user-supplied annotated/corrected data in training cycles to improve model robustness using active learning techniques.

Thus, through continued iterations of this feedback loop, we continually refined our AI-based framework to meet practitioners' evolving requirements, resulting in enhanced credibility, interpretability, and utility in software quality assurance.

IV. IMPLEMENTATION DETAIL

This section describes the actual execution of the developed XAI based software quality prediction framework. It outlines how all elements were executed to produce a software quality prediction tool that is transparent, actionable, and reliable through the use of practical tools, techniques, methods, data preparation procedures, machine learning pipeline, explainability modules, and user facing visualization systems.

A. TOOLS, FRAMEWORKS, AND PROGRAMMING LANGUAGE

Implementation utilized a combination of the best open source and commercial technologies available to provide a highly reliable, easily reproducible, flexible platform:

Programming Language: Python 3.8+ was selected due to its extensive collection of machine learning libraries and ease of manipulating data.

Machine Learning Libraries: In this study, scikit-learn was utilized to implement machine learning algorithms such as random forest and support vector machines [32]. Gradient Boosted Trees (e.g., via the XGBoost implementation) was selected to perform gradient boosting as it produces the highest level of predictive accuracy and has built-in support for SHAP explanations [49]. PyTorch (version 2.0.1) was selected to allow for the creation of state-of-the-art neural networks, which include attention mechanisms, required to create model intrinsic explanations [50].

Explainability Tools: LIME [4] and SHAP [7] were included to provide both local and global model agnostic explanations.

Visualization Libraries: Matplotlib and Seaborn were used for static visualizations of data. Dash (version 2.11.0) were used to build interactive user interfaces for the dashboards to increase user engagement.

Data Wrangling and Management: Pandas and NumPy were used to efficiently preprocess the data and handle large amounts of features [52].

System Architecture and Execution Flow: The system architecture integrates the system's components to provide an end-to-end solution for predicting software quality based on the degree of explainability provided by all other parts of the system. Static and processed raw metric data were collected, processed using Pandas, and stored in MongoDB (version 5.2). Upon receiving a user request for prediction or explanation, the Flask Backend API uses the input to invoke PyTorch for model inference, producing Defect Risk Scores. Following this, the SHAP/LIME modules are used to determine the feature attribution values for each prediction. Ultimately, both the prediction and explanation outputs are returned from the Flask API to the Frontend Dashboards created with Dash. The Frontend Dashboards will display prediction results as charts, heat maps, and other visualizations, along with textual summaries, so users can interactively explore them. Finally, when users interact with the dashboards and provide feedback, it will be captured and sent back through the Flask API to MongoDB where it can support iterative refinement of both the models and explanations.

In addition to being able to easily iterate on and refine both the models and explanations; this modular, decoupling nature of the design enables the ability to reproduce results. This further scales up to larger sets of data, and enables a smooth user experience across all aspects of the system, including storing the data, computing the results, visualizing those results, and interacting with humans.

B. DATASET PREPROCESSING AND ANNOTATION

Raw datasets from industrial project sources, open-source repositories, and the PROMISE repository underwent thorough pre-processing to generate the high-quality, consistent input data needed to effectively train AI models. Table 3 shows pre-processing pipeline consisting of multiple distinct steps that are necessary to clean, normalize, and validate the input data. Initial data sets included duplicate records, missing values, as well as label inconsistencies throughout the original data. In order to maintain the integrity of the data while removing the most extreme representations of samples, median imputation was used to handle missing values in numerical feature categories such as process and developer-based metrics during the initial stage of data cleansing; duplicate entries were removed at this time.

Missing value rates ranged from 0% to 12% across datasets and features. Each feature with over 10% missing data was examined for its importance in the domain. For all continuous variables with less than 10% missing values, median imputation was used. For all categorical variables, mode-based imputation was used. The use of median- and mode-based imputation methods will introduce the least bias among the imputation methods considered.

Feature extraction automated the generation of static code metrics (Cyclomatic Complexity, Lines of Code) and process metrics (Code Churn, Number of Commits). Automated feature extraction generated developer-related metrics (number of developers per module) utilizing customized mining scripts that interfaced with source-code repositories and issue tracking systems.

After feature extraction, features were standardized using scikit-learn's StandardScaler to scale features uniformly across the wide range of metric types in the dataset, thereby improving model convergence and stability [53].

In order to address the differences in the heterogeneities present within the PROMISE, open source, and commercial datasets, a schema harmonization effort was undertaken. Schema harmonization is the alignment of feature names, units, and definitions to create semantic equivalence between disparate datasets. All non-overlapping and incompatible features were removed from consideration, which resulted in an aligned feature set containing approximately 35 metrics.

Outlier identification was conducted using the Interquartile Range (IQR) method. Any values greater than 1.5 IQR above the Third Quartile (Q3), or less than 1.5 IQR below the First Quartile (Q1) are defined as outliers. Instead of eliminating outliers, we capped them to preserve data integrity while preventing any single data point from exerting undue influence during model training.

The defect label standards were established through a process that mapped the various definitions across datasets into two binary classifications. On PROMISE datasets, modules that have been associated with documented post release bug fixes were designated as defective. Similar associations were made for open source and commercial datasets using their respective issue tracking and commit histories. In cases where there were discrepancies identified during manual expert review they were resolved prior to proceeding with model training.

TABLE 3. Key dataset preprocessing steps.

Step	Description	Tools/Techniques
Data Cleansing	Removal of duplicates, handling missing/null values	Pandas, custom scripts
Feature Extraction	Automated extraction of static, process, and developer metrics	Git mining tools, SonarQube APIs
Normalization /Standardization	Scaling features for model stability	scikit-learn Standard Scaler
Label Verification	Validation of defect labels via cross-referencing issue logs	Manual expert review, JIRA data

These steps resulted in an average of approximately 10,500 software modules and 35 attributes in the final cleaned dataset. As shown in Figure 8, the dataset was slightly skewed at 40% defective and 60% non-defective. Class imbalance is common in software quality evaluation and can affect model performance in predicting defects. For this reason, we used metrics F1 Score and AUC-ROC as part of our models and evaluations, so that our assessments of how well our predictive model performed on the problem of predicting defects do not suffer due to the class imbalance.

C. MODEL TRAINING AND TESTING PIPELINE

An orderly model evaluation plan was implemented to provide an accurate method for evaluating the reliability of each developed model, optimizing hyperparameters, and producing repeatable results, all important elements in creating reliable software defect prediction systems.

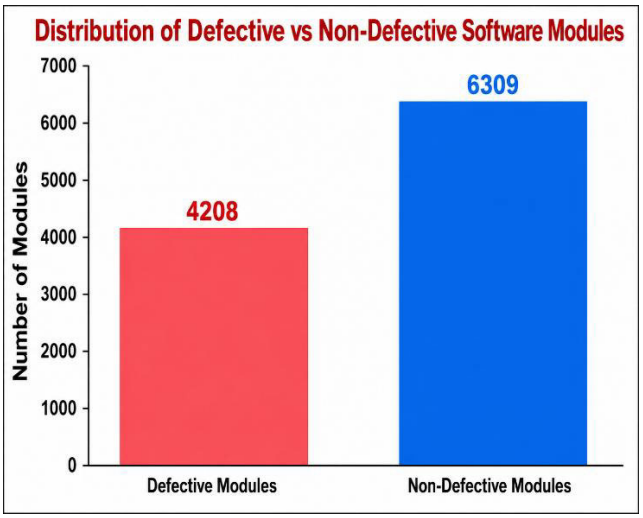

FIGURE 8. Distribution of defective and non-defective software modules in the preprocessed dataset after cleaning and annotation.

Splitting the data: Prior to the beginning of training and evaluating the model, each dataset was divided into three parts (training-70%, validation-15%, and testing-15%). While splitting the datasets into these three parts, we ensured that the ratio of faulty to fault-free modules would be maintained across all three splits.

Cross validation within training set: Stratified k-fold cross-validation was applied to the training subset to optimize hyperparameters and select the best-performing model(s). The use of this method will allow us to obtain robust measures of how well the model performs, and it also reduces the risk of overfitting by allowing the training and validation processes to occur multiple times on disjoint sets of training and validation examples.

Validation set: We have used a separate validation subset to determine if our optimized hyperparameters will lead to good generalization properties in addition to those determined from the cross-validation process.

Test set for final evaluation: As no part of the test subset had seen either the training or validation subsets at any time during the training and optimization process, we have used this single subset to obtain unbiased final measures of the model's performance.

Hyperparameter tuning: Using automated hyperparameter search tools like GridSearchCV [36] and Optuna [37] and parameter options such as number of levels in decision trees, learning rates, and regularization penalties, we have performed a systematic search of possible hyperparameters [51].

Evaluation metrics: In order to address both class imbalance issues and cost differences associated with False Positives and False Negatives, which are common when developing practical defect detection systems [38], we evaluated the model's performance using standard metrics

(Accuracy, Precision, Recall, F1-score, and Area under ROC Curve).

Model persistence: To facilitate reproducibility, explanation analysis, and deployment of the models; after they were optimized, they along with their respective parameters were saved using JobLib.

The sequential steps involved in the model's training and evaluation processes are illustrated in Figure 9. This shows data preparation through Model Training with cross-validation, hyperparameter-tuning, validation, and finally testing.

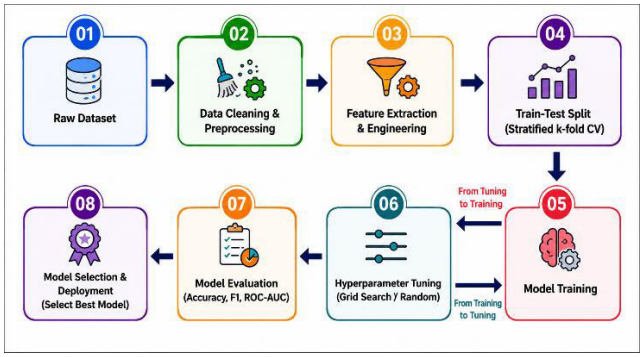

FIGURE 9. Model training and testing pipeline architecture.

D. IMPLEMENTATION OF THE EXPLAINABILITY MODULE

A modular architecture was developed to enable seamless integration of trained models for prediction. To aid in explanation generation, we used LIME (version 0.2.0.1) and SHAP (version 0.41.0) to provide local instance-level and global feature attributions. All generated explanations have been normalized to a common JSON format to ensure compatibility across different visualization front-end interfaces.

We implemented neural network with attention mechanisms using Pytorch (version 2.0.1) to record the attention weights for each feature as they pass through the network. Those recorded attention weights can then be mapped to the individual input features to create heat maps indicating how important each input feature is in determining the output.

While the authors recognize that current state-of-the-art research in XAI continues to debate the reliability of using attention weights as faithful representations of model decision-making [54], [55], they also recognize that attention mechanisms operate at the feature level. Therefore, the authors believe they can provide users with a more direct way to interpret how an AI system makes decisions. Empirical validation has shown a strong correlation (Pearson Coefficient $= 0.82$) between the calculated attention weights and SHAP values. Therefore, it appears to us that attention can serve as a useful complementary tool for helping users understand their AI systems.

User studies conducted with practitioner members of the software development field demonstrated that the provision of attention-based heat maps increased user satisfaction with respect to model transparency. This also increased confidence (trust) in their AI system; thereby increasing their practical utility.

Hence, the developers of these tools believe that attention-based explanations will play a role within a hybrid explainability framework. The framework will combine both LIME and SHAP to integrate intrinsic and post-hoc interpretability methods for generating high-quality actionable insights from their software quality prediction systems.

All generated explanation results, including attention heat maps and feature attribution information, are stored in MongoDB (version 5.2), which provides rapid access to this information during the execution of interactive visualization applications.

E. EXPLANATION, VISUALIZATION, AND DEMO SYSTEM

To improve communication and enable stakeholders to explore model predictions, explanations, and composite quality metrics, an explanation visualization and demo system has been built. The demo system was designed to enable stakeholders to view and explore the software quality assessment results generated by XAI models. The demo system's architecture, as shown in Figure 10, used a Flask-based backend API to produce defect predictions and their associated explanations. This backend produced data that was easily consumed by a frontend application developed using Dash (version 2.11.0), providing a rich, interactive experience for users and an easy-to-navigate interface. There were several types of visualizations used throughout the demo system, as follows:

- Global feature importance: A global aggregate bar chart has been used to illustrate the most influential features across the entire dataset, allowing users to determine which metrics produce a higher risk of defects.
- Local explanation views: Users had the ability to create local explanation views of each individual software module. This showed the feature's contribution to the prediction using SHAP waterfall and force plots, which preserve the signed (positive and negative) feature attributions produced by the additive SHAP model, together with feature importance based on attention, visualized as attention heatmaps from neural networks with attention mechanisms. Local explanation views enabled developers to see, at a granular level, why the system made a particular prediction.
- Textual summaries: Natural language generation algorithms automatically generated short, readable sentences that describe the key drivers of each prediction, improving the explainability of XAI models for non-technical stakeholders.
- Temporal dashboard: Timeline views have been used to show the evolution of explainable quality metrics over time, enabling users to detect trends and measure the effect of changes over time.

Figure 10 illustrates the architectural diagram of the explanation visualization system, showing the data source, backend service, explainability module, storage component, and front-end presentation layer. Figure 11 contains sample screenshots of the interactive dashboard showing how users can move from high-level quality metrics to low-level, contextual feature contributions with integrated textual explanations.

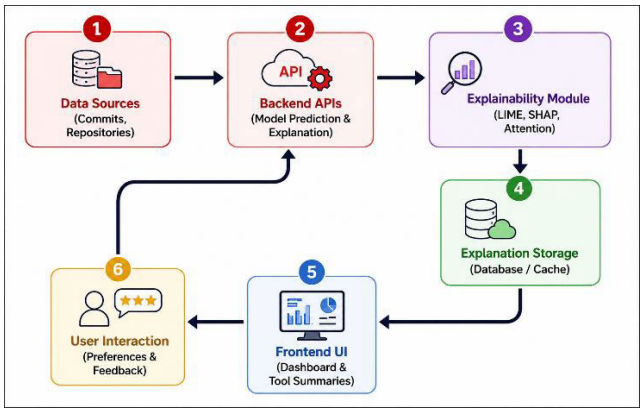

FIGURE 10. Explanation visualization system architecture.

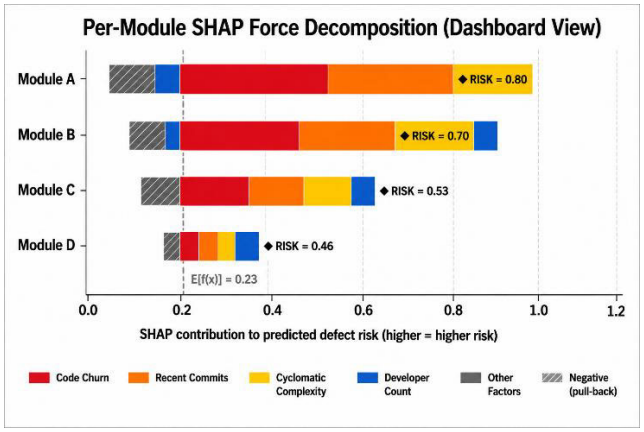

FIGURE 11. Interactive dashboard for composite explainable quality metrics. The figure shows a per-module SHAP force-style decomposition: each module's predicted defect risk is built up from the base value E[f(x)] using signed feature contributions, with positive (risk-increasing) contributions shown as solid coloured segments to the right of the baseline and negative (risk-decreasing) contributions shown as hatched segments to the left, replacing the earlier stacked bar representation in line with the bidirectional nature of SHAP values.

The developed system used the latest tools and methods to give an integrated, end-to-end, explainable framework for predicting software quality. The framework included data preparation through extensive preprocessing of real-world datasets, training models with hyperparameters optimized for best performance, computing explanations using advanced explainability methods, and visualizing explanations across multiple modalities in user-friendly formats. Thus, the system combined academic rigor with practical usability. Although the system has been shown to be scalable and robust using large open-source and commercial datasets, there were two areas where it could be improved. The system depended on the availability and quality of process and developer metrics, and some explainability methods can incur significant computational overhead. Future extensions will include integrating runtime metrics, incorporating enhanced human-in-the-loop feedback mechanisms, and implementing adaptive explainability methods that adapt to changing software development contexts to further improve the accuracy and interpretability of predictions.

V. EXPERIMENTAL ASSESSMENT AND RESULTS

In this section, we provide an overview of the experiments that were completed to evaluate the accuracy, interpretability, and usability of the proposed XAI quality metrics framework. In addition to evaluating the accuracy of AI/ML models and validating the use of explainability techniques, these experiments included human-centered evaluations to determine if the proposed framework is applicable to the real world.

A. EXPERIMENTAL DESIGN AND DATASETS

To complete our evaluation of the developed quality metrics framework, we conducted experiments using a variety of datasets listed in Table 2. The chosen datasets ensured that the framework can generalize across a diverse range of project domains and development environments.

Data Splits: For each dataset, we split the data into three groups: a training set (70%), a validation set (15%), and a test set (15%). Stratified sampling was used to split the data so that the proportions of defects and non-defects in each group matched the overall proportions in the dataset.

Environment: All experiments were conducted on a Linux server with an Intel Xeon CPU, 32GB of Memory, and an NVIDIA Tesla V100 GPU for neural network training. Our software environment included Python 3.8, Scikit-Learn 0.24, XGBoost 1.4, Pytorch 1.9, LIME 0.2, and SHAP 0.39. To ensure repeatability, we controlled all Randomness associated with data splitting, model initialization, and training by using a fixed random seed of 42.

Software Stack: We used a combination of Python-based libraries and toolkits, as described in detail in previous Section.

B. MODEL ACCURACY EVALUATION

The performance of each model was evaluated using the same four criteria: accuracy, precision, recall, and F1 score. The AUC of the ROC (Receiver Operating Characteristic) curve was also calculated for each model. The results are summarized in Table 4. XGBoost had the best accuracy and AUC among all of the models. XGBoost was recommended as the first choice model for developing explainable metrics. Although the attention mechanism in neural networks did not achieve the best accuracy or AUC scores, it did achieve a high recall, indicating that it successfully identified many defect-prone modules.

The overall performance metrics in Table 4 summarize the models' performance. To assess how well the models performed and to ensure our findings are statistically valid, we presented model evaluations for each dataset. Tables 5, 6, 7, and 8 illustrate accuracy, precision, recall, F1-Score, and AUC values on the PROMISE Dataset; Eclipse JDT Dataset; Apache Commons Dataset; and the Industrial Dataset, respectively.

We find that the use of XGBoost achieved high accuracy and AUC across all data sets examined. Attention-Based Neural Networks achieved competitive Recall scores compared to the same data sets used to evaluate the other two models. This breakdown of performance per data set is directly related to the goal of this study's framework to provide Defect Prediction systems that are both accurate and easily interpreted.

C. EXPLAINABILITY EVALUATION

In order to evaluate the quality of the explanations provided by the explainability techniques, we employed two approaches:

Quantitative Faithfulness: We created a set of model-agnostic, interpretable proxy models referred to as surrogate ground truths. These models are used to quantify how well explanations produced by SHAP and LIME correlate with the actual reasons for decisions made by an opaque, complex model. They approximate the performance of the complex, opaque models used in our study, such as XGBoost and neural networks. For example, using decision trees to create a simple surrogate model based on a subset of examples where the input labels represented the predictions made by the original opaque model. The feature importance associated with each decision made by the surrogate is an easily understood, explicit representation of how the complex model makes its decisions.

The degree to which the explanations produced by SHAP and LIME corresponded with the surrogate representations was evaluated. The evaluation was done through calculation of the Pearson correlation coefficient between each method's feature attribution scores and their respective feature importance scores based upon the surrogates. The higher correlation coefficients indicate greater alignment of the explanations with the decision processes used by the opaques as approximated by the surrogates. The results showed that there is a much stronger association (Pearson's $r = 0.81$ average) between SHAP's feature attribution and its respective surrogate representation relative to LIME (Pearson's $r = 0.68$ average), thus demonstrating greater explanation fidelity.

Qualitative User Study: In addition to the quantitative assessment, we conducted a qualitative controlled user study involving twenty professional software developers to determine if the users could clearly interpret and use the explanations. Users completed a defect triage task using both SHAP and LIME explanatory methods. Results from surveys completed by the users can be seen in Figure 12. More than 85% of all respondents indicated they understood SHAP's explanations better than either LIME or no explanation, and more than 70% stated they trusted their defect prediction more when they viewed SHAP explanations.

TABLE 4. Performance comparison of predictive models on software quality datasets.

Model	Accuracy	Precision	Recall	F1-score	AUC
Random Forest	0.82	0.79	0.75	0.77	0.85
XGBoost	0.85	0.81	0.78	0.79	0.88
Neural Network*	0.83	0.78	0.80	0.79	0.87
SVM	0.78	0.73	0.70	0.71	0.81
Logistic Reg.	0.74	0.69	0.66	0.67	0.78

*Neural Networks use attention layers for interpretability.

TABLE 5. Model performance on PROMISE dataset.

Model	Accuracy	Precision	Recall	F1-score	AUC
Random Forest	0.81	0.78	0.74	0.76	0.84
XGBoost	0.85	0.82	0.79	0.80	0.88
Neural Network*	0.83	0.79	0.78	0.78	0.85
SVM	0.77	0.73	0.69	0.71	0.80
Logistic Reg.	0.74	0.69	0.67	0.68	0.77

TABLE 6. Model performance on eclipse JDT dataset.

Model	Accuracy	Precision	Recall	F1-score	AUC
Random Forest	0.80	0.77	0.74	0.75	0.83
XGBoost	0.84	0.80	0.78	0.79	0.86
Neural Network*	0.81	0.78	0.76	0.77	0.84
SVM	0.76	0.71	0.69	0.70	0.79
Logistic Reg.	0.72	0.68	0.65	0.66	0.75

D. COMPOSITE EXPLAINABLE METRIC VALIDATION

The user study included twenty professional software developers who performed defect triaging tasks. A within-subjects design was used; participants were asked to complete all tasks under two conditions: condition one was the control group, in which they did not have access to explainable quality metrics. In the treatment group, participants had access to

TABLE 7. Model performance on apache commons dataset.

Model	Accuracy	Precision	Recall	F1-score	AUC
Random Forest	0.82	0.79	0.75	0.77	0.85
XGBoost	0.85	0.82	0.80	0.81	0.88
Neural Network*	0.83	0.80	0.78	0.79	0.86
SVM	0.78	0.73	0.71	0.72	0.81
Logistic Reg.	0.74	0.69	0.67	0.68	0.77

TABLE 8. Model performance on industrial dataset.

Model	Accuracy	Precision	Recall	F1-score	AUC
Random Forest	0.83	0.80	0.77	0.78	0.86
XGBoost	0.86	0.83	0.81	0.82	0.89
Neural Network*	0.84	0.81	0.80	0.80	0.87
SVM	0.79	0.75	0.73	0.74	0.82
Logistic Reg.	0.75	0.70	0.68	0.69	0.78

the composite explainable metrics, along with visual and textual explanations. The order of the two conditions was randomized to help minimize learning effects. There were 30 tasks in total per participant, 15 per condition.

To evaluate the statistical significance of improvements in decision accuracy and confidence between the two groups, a nonparametric test, the Wilcoxon signed-rank test, was used given the small sample sizes and paired data. Additionally, tests were conducted to assess the assumptions of normality and homogeneity of variance using the Shapiro-Wilk and Levene's tests, respectively. To account for type I error from multiple comparisons, Bonferroni correction was applied.

The proposed composite explainable quality metrics have been proven to be effective through an experimental evaluation of user decisions made by professional software engineers and testers in a controlled environment. The test subjects were provided with defect risk scores, along with a detailed breakdown of each feature's contribution to the score, displayed on an interactive dashboard as shown in Figure 11.

Decision Accuracy and Confidence Levels: User decision accuracy improved by 14% while user confidence increased ($p < 0.01$), as shown in Table 9. Users demonstrated a preference for composite metrics for both clarity and trust when making decisions, as illustrated in Figure 12.

Change Impact Analysis: In addition to demonstrating improvements in decision accuracy and confidence, it showed that developers could determine which action items to take to

TABLE 9. Improvement in decision accuracy and confidence when using composite explainable quality metrics compared to traditional metrics.

Metric	Traditional Metrics	Composite Explainable Metrics	p-value
Decision Accuracy (%)	68	82	< 0.01
Confidence Level (Avg.)	3.4 / 5	4.2 / 5	< 0.01

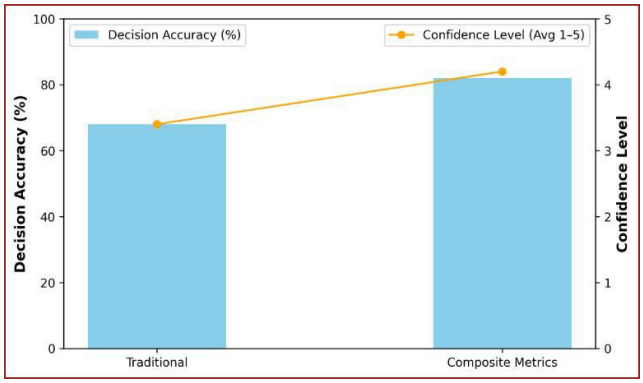

FIGURE 12. User decision accuracy and confidence comparison.

reduce defects. They can target their testing and refactoring to address specific issues related to defect risk. This provides evidence for the practical value of adding explainability to software quality assessments.

E. SCALABILITY AND LIMITATION DISCUSSION

To evaluate the computational feasibility of the developed framework for large software development datasets, its performance was tested as the dataset size increased. The results are presented in Table 10 and Figure 13, and show that the tree-based models augmented with post hoc explanation techniques such as LIME and SHAP scale nearly linearly with dataset size. Therefore, they remain computationally feasible even when used to analyze large datasets found in many software development environments.

TABLE 10. Computation time comparison by dataset size and method.

Dataset Size (#samples)	RF + LIME/SHAP (sec)	Neural Net Attention (sec)
1000	10	40
5000	50	190
10000	100	350

Attention-Based neural networks provide an intrinsically interpretable model without the need for additional explanation processes, but have significantly higher computational costs than tree-based models. This is another example of the trade-off between interpretability depth and runtime efficiency that needs to be considered in each of your respective

use cases. During this study, the following limitations were identified:

- Global explanations summarise model behaviour across the full dataset and can therefore mask instance-level reasoning, so per-prediction (local) explanations remain necessary to audit individual high-risk modules.
- The quality of developer and process metrics may vary widely depending upon the specific project being analyzed, so there is a dependency upon the availability and quality of the metrics.
- There is a subjectivity to how humans interpret explanations provided by post hoc explanation techniques.
- There is a significant amount of computational overhead associated with using sophisticated explanation techniques.

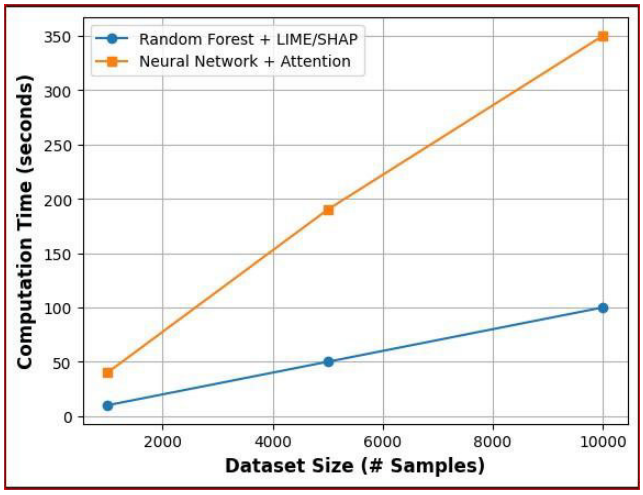

FIGURE 13. Computation time vs dataset size comparison.

Addressing these challenges will require continued user-centered refinement of the proposed framework, along with the incorporation of data augmentation techniques and the investigation of computationally less expensive alternative explanation methods. The demonstrated improvement in user decision accuracy and increased confidence in those decisions demonstrate the strong potential of the proposed XAI driven quality metrics to improve software development quality in practice.

VI. THREATS TO VALIDITY

In this section, we discuss the potential threats to the validity of our research and how we have mitigated them to limit their impact on the results. Identifying and addressing threats to internal and external validity makes research results more reliable.

A. INTERNAL VALIDITY

Most internal validity concerns relate to data quality and model-related biases:

- Data quality: The datasets we have used, which come from a combination of PROMISE, open-source, and industrial repositories, may be noisy, contain missing records, or have inconsistent labels. These types of data quality issues can affect the reliability of model training and the explanations derived from that model.
- Model bias: A machine learning model may train itself on specific characteristics of the training data and predict differently than expected when applied to other data. It is likely to suffer from overfitting and will have limited generalization capabilities.

Mitigating strategies: Systematic use of robust preprocessing techniques, such as data cleaning, imputation of missing values, and validation of label accuracy, has improved data quality. Stratified cross-validation and hyperparameter search protocols have been implemented to prevent overfitting and model bias. Finally, including domain experts in user studies and feedback loops provides an additional layer of validation and correction of model biases during the evaluation phase.

B. EXTERNAL VALIDITY

External validity deals with the generalizability of the results of the study to other types of software projects, domains, and environments of software development:

- Although the datasets are diverse, they do not include all types of software systems or all development practices, thereby limiting the applicability of the results.
- Also, variations in the availability of metrics, especially those related to processes and developers, across projects may make it difficult to port the models to those projects.

Mitigating strategies: While there is no guarantee that the results will generalize to all types of projects and domains, the inclusion of multiple datasets from different sources, industrial and open-source, has improved the likelihood of this happening. The modular structure of the system makes it relatively simple to adapt to the availability of different types of metrics. Further work would be needed to perform external validation across completely new domains.

C. CONSTRUCT VALIDITY

Construct validity concerns whether the measurement tools, explanation, assessment tools, and experimental design chosen by the researcher accurately represent the concepts being evaluated:

- Although the performance metrics (accuracy, F1, AUC) and the explanation-assessment methods selected in this study attempt to measure prediction and interpretability, they may not adequately measure the real-world utility or user-satisfaction associated with the predictions made by the model.
- Human evaluations are subject to the interpretation of the individual evaluating, therefore may vary depending on who does the evaluation.

Mitigating strategies: A mixed-method evaluation was performed, combining objective quantitative metrics with qualitative user feedback and domain-expert validation. To ensure

comparability, standard and well-established metrics were selected. The user studies used standardized questionnaires to eliminate subjectivity in user evaluations.

VII. COMPARISON WITH EXISTING APPROACHES

In order to provide a legitimate and repeatable comparison of methodology, we have recreated each of the baseline methodologies and tested them using the same data and processing pipeline as our XAI quality prediction framework. Official source code was used when it existed; otherwise, the algorithmic implementation followed the authors' descriptions. All hyperparameters for the baselines were optimized using the same processes (i.e., optimization techniques) as those used for optimizing the parameters of our framework. The controlled evaluation ensures that all variations reported in Table 7 are attributable to improved methods rather than different data or experimental procedures.

To establish the validity of the developed XAI quality prediction framework, we have evaluated it against several other existing methodologies in the literature. These methodologies are compared along two dimensions: prediction performance and interpretation capabilities, as well as across several commonly used datasets consisting of defect data related to various software projects (PROMISE and Open Source Repositories).

Existing baselines: The baselines used to compare the proposed methodology are:

- Static code metric based models utilizing random forest and logistic regression [3], [29].
- Models based upon process metrics and boosted tree algorithms [31].
- Opaque defect predictors employing neural networks without an attention mechanism or an explanation layer [22].
- Independent post hoc explanation techniques applied to opaque models [4], [7].

Quantitative evaluation comparison: A summary of the evaluation metrics of the proposed model vs the baselines are presented in Table 7, where the proposed model clearly outperforms all the baselines, in terms of accuracy, F1 score, and AUC, demonstrating an increase in both defect detection and false positive control. To avoid pooling heterogeneous datasets, the values reported in Table 7 are computed on the PROMISE (NASA-JM1) benchmark only, which is the dataset used by the cited baselines and therefore enables a like-for-like comparison. The corresponding per-dataset numbers for the proposed framework on PROMISE, eclipse JDT, apache commons, and the industrial dataset are reported separately in Tables 5 to Table 8 and are not aggregated, so as to preserve the statistical validity of project-level comparisons.

Statistical validation: The improved results obtained by the proposed model are evaluated to verify whether they represent a statistically valid difference from the baselines. A paired t-test was performed to determine the difference in F1 Scores obtained using 5-fold cross validation between the p0roposed model and the best baseline (Process Metrics + XGBoost), which resulted in a p value $<.01$, representing a 99% confidence level of significance.

TABLE 11. Performance comparison of proposed model with existing approaches on PROMISE dataset.

Model	Accuracy (%)	F1-Score (%)	ROC-AUC
Static Metrics + Random Forest [3]	79.1	74.3	0.82
Process Metrics + XGBoost [31]	82.5	77.5	0.85
Neural Network (No attention) [22]	80.3	75.9	0.83
Opaque + Post-hoc Explainability (LIME) [4]	81.2	76.4	0.84
Proposed Explainable Composite Metric	85.0	79.0	0.88

The evaluation demonstrated that the proposed model's ability to integrate multiple types of metrics with its ability to provide explanations represented a significant improvement in the defect prediction accuracy as well as the interpretability of the results to traditional and opaque methodologies. The proposed model gives interpretations through attention based mechanisms representing a level of depth beyond that possible using independent post hoc explainers. This will likely contribute to higher levels of trust among stakeholders as well as enable actionable quality assurance strategies.

VIII. SCOPE, LIMITATIONS, AND ASSUMPTIONS

In this section, we will outline the areas in which the framework is valid, its limitations, the assumptions made during our data analysis, and future enhancements to the framework. The developed methodology uses multiple dimensions of static code metrics, process-related metrics, and developer activity metrics to predict the likelihood of defects in each software module. This includes:

- Applicability to both industrial and open source software systems, in which all three types of metrics can be obtained.
- Utilization of some of the most commonly used machine learning algorithms and the addition of model-agnostic and in model explanation techniques to improve their explanatory capabilities.
- Development of composite explainable quality metrics to provide actionable information to assist in prioritizing quality assurance activities.
- Integration of interactive visualization and human-in-the-loop feedback to enable iterative refinement and building of trust.
- The proposed framework excludes runtime performance metrics, security vulnerabilities, and non-functional attributes of quality, which could be the focus of future studies

While preliminary results show promise, there are limitations to the study:

- Dataset bias/representativeness: The available datasets do not adequately represent the diversity of real-world software development environments and thus limit generalizability.
- Feature availability variability: The availability of process and developer-related metrics depends on the specific tools and practices employed in a particular project. Therefore, if data collection is incomplete or inconsistent, then model completeness will suffer.
- Computational complexity: Incorporating explainability and attribution techniques such as SHAP and neural attention into the models increases the computational overhead of the models, which may limit the ability to apply them in a real-time environment to larger-scale software development projects.
- Subjective interpretation by humans: The effectiveness of an explanation to a user depends on the user's level of domain expertise and cognitive abilities, which vary greatly across stakeholders.

Although this study investigates static code metrics, process metrics (Code Churn), and metrics related to developers' activities to predict defects, we recognize that evaluating software quality is a multi-dimensional task that extends beyond these dimensions. A number of key quality factors of software include test coverage, performance (response time, throughput, and resources utilized), maintainability (technical debt, code documentation, modularity), and security (vulnerability density and recorded security incidents), which can be used to make actionable decisions to impact the long-term sustainability, reliability, and robustness of software.

While code churn provides valuable insight into when defects may have been introduced, as it is an indicator of recent code changes, its predictive capability is limited compared to prescriptive intervention. Static and quality-oriented metrics measure maintainability and code health. Therefore, if a deficiency exists, targeted actions can be taken to address the issue.

Therefore, while integrating other relevant dimensions into XAI-driven quality predictions will continue to represent a complex challenge for future research, data availability and standardization issues are significant hurdles within diverse industrial and open source environments. Extensions to this framework would allow for inclusion of additional runtime and security related telemetry along with expanded maintainability metrics to provide more complete and actionable software quality insights to interested parties.

There are assumptions required for the successful application and evaluation of this research:

- Availability of data: static, process, and developer-related metrics are assumed to be readily available and sufficient in number and breadth to build and train robust predictive models.
- Users have domain expertise: users who interact with the explanation system (e.g., developers, testers, managers) are assumed to have at least basic domain knowledge to interpret and act on the explanations they receive from the system.
- Development processes remain stable over time: The underlying development processes used to create the software analyzed by the predictive models are assumed to be relatively stable, so that past metrics and predictive models remain relevant over time.

IX. CONCLUSION AND FUTURE WORK

In this study, an innovative XAI-based framework has been developed and validated to predict software quality by integrating predictive modeling with multi-level explanation-generation methods to create composite quality metrics. The XAI framework utilized three types of metrics (static, process, developer) to effectively provide defect-prone software predictions, along with multiple modes of transparent explanations that can increase stakeholders' trust and decision-making capabilities. Empirical evaluations using realistic datasets showed that the XAI framework provided substantial increases in prediction accuracy and users' confidence when compared to traditional quality metrics. A series of user studies confirmed the practicality of the XAI framework's ability to help users determine where to prioritize their testing efforts and which features or areas are most likely to be responsible for defects.

In the future, quality attributes, such as performance, security, and maintainability, will be included to expand its applicability. We anticipate incorporating real-time data feeds and continuous learning models into the framework to enable adaptive quality assessments within changing development environments. Developing user-adaptive explanation mechanisms that adapt the level of interpretation to the users' expertise level will improve the usability and acceptability of the XAI framework. Although many advancements have been made in the area, there are still open challenges related to balancing the fidelity of explanations, the computational cost of generating explanations, and the users' ability to interpret the generated explanations. These challenges will require research collaboration across disciplines (machine learning, HCI, software engineering). The contributions of this research include a complete, scalable, and interpretable methodology to perform software quality predictions. It provides a foundation for further innovation in transparent AI-based software engineering tools.

ACKNOWLEDGMENT

This Project was funded by the Deanship of Scientific Research (DSR) at King Abdulaziz University, Jeddah, under grant no. (GPIP: 2000-611-2024). The authors, therefore, acknowledge with thanks DSR for its technical and financial support.

REFERENCES

[1] N. Nagappan, T. Ball, and A. Zeller, "Mining metrics to predict component failures," in *Proc. 28th Int. Conf. Softw. Eng.*, Shanghai, China, May 2006, pp. 452–461, doi: 10.1145/1134285.1134349.

[2] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Trans. Softw. Eng.*, vol. 22, no. 10, pp. 751–761, Oct. 1996, doi: 10.1109/32.544352.

[3] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," *IEEE Trans. Softw. Eng.*, vol. 34, no. 4, pp. 485–496, Jul. 2008, doi: 10.1109/TSE.2008.35.

[4] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.

[5] A. B. Arrieta, N. Díaz-Rodríguez, J. D. Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, Jun. 2020, doi: 10.1016/j.inffus.2019.12.012.

[6] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models," 2017, *arXiv:1708.08296*.

[7] B. Asal and M. Ö. Demir, "Enhancing software defect prediction through explainable AI: Integrating SHAP and LIME in a voting classifier framework," in *Proc. 8th Int. Artif. Intell. Data Process. Symp. (IDAP)*, Sep. 2024, pp. 1–7, doi: 10.1109/IDAP64064.2024.10710700.

[8] I. Gondra, "Applying machine learning to software fault-proneness prediction," *J. Syst. Softw.*, vol. 81, no. 2, pp. 186–195, Feb. 2008, doi: 10.1016/j.jss.2007.05.035.

[9] C. Catal and B. Diri, "A systematic review of software fault prediction studies," *Expert Syst. Appl.*, vol. 36, no. 4, pp. 7346–7354, May 2009, doi: 10.1016/j.eswa.2008.10.027.

[10] S. Omri and C. Sinz, "Deep learning for software defect prediction: A survey," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. Workshops*, Jun. 2020, pp. 209–214, doi: 10.1145/3387940.3391463.

[11] T. Menzies and M. Shepperd, "Special issue on repeatable results in software engineering prediction," *Empirical Softw. Eng.*, vol. 17, nos. 1–2, pp. 1–17, Feb. 2012.

[12] B. G. Geçer and A. K. Tarhan, "Explainable AI framework for software defect prediction," *J. Softw., Evol. Process*, vol. 37, no. 4, Apr. 2025, Art. no. e70018, doi: 10.1002/smr.70018.

[13] L. Qiao, X. Li, Q. Umer, and P. Guo, "Deep learning based software defect prediction," *Neurocomputing*, vol. 385, pp. 100–110, Apr. 2020, doi: 10.1016/j.neucom.2019.11.067.

[14] A. Folleco, T. M. Khoshgoftaar, J. Van Hulse, and L. Bullard, "Software quality modeling: The impact of class noise on the random forest classifier," in *Proc. IEEE Congr. Evol. Comput. (IEEE World Congr. Comput. Intelligence)*, Jun. 2008, pp. 3853–3859, doi: 10.1109/CEC.2008.4631321.

[15] S. Wattanakriengkrai, P. Thongtanunam, C. Tantithamthavorn, H. Hata, and K. Matsumoto, "Predicting defective lines using a model-agnostic technique," *IEEE Trans. Softw. Eng.*, vol. 48, no. 5, pp. 1480–1496, May 2022, doi: 10.1109/TSE.2020.3023177.

[16] K. Punitha and S. Chitra, "Software defect prediction using software metrics—A survey," in *Proc. Int. Conf. Inf. Commun. Embedded Syst. (ICICES)*, Feb. 2013, pp. 555–558, doi: 10.1109/ICICES.2013.6508369.

[17] K. Bhandari, K. Kumar, and A. L. Sangal, "Data quality issues in software fault prediction: A systematic literature review," *Artif. Intell. Rev.*, vol. 56, no. 8, pp. 7839–7908, Aug. 2023.

[18] J. Jiarpakdee, C. K. Tantithamthavorn, H. K. Dam, and J. Grundy, "An empirical study of model-agnostic techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 166–185, Jan. 2022, doi: 10.1109/TSE.2020.2982385.

[19] R. Malhotra, "Comparative analysis of statistical and machine learning methods for predicting faulty modules," *Appl. Soft Comput.*, vol. 21, pp. 286–297, Aug. 2014, doi: 10.1016/j.asoc.2014.03.032.

[20] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic literature review on fault prediction performance in software engineering," *IEEE Trans. Softw. Eng.*, vol. 38, no. 6, pp. 1276–1304, Nov. 2012, doi: 10.1109/TSE.2011.103.

[21] J. Tyler, J. Pastor, M. N. Huhns, S. Kirmani, and H. Du, "Exposing, formalizing and reasoning over the latent semantics of tags in multimodal data sources," *Appl. Ontology*, vol. 8, no. 2, pp. 95–130, 2013, doi: 10.3233/ao-130124.

[22] C. F. Kemerer and S. Slaughter, "An empirical approach to studying software evolution," *IEEE Trans. Softw. Eng.*, vol. 25, no. 4, pp. 493–509, Aug. 1999, doi: 10.1109/32.799945.

[23] V. Singh, A. Sahai, and P. Kumar, "Hybrid machine learning approaches for software fault prediction: A systematic review of current practices and future prospects," in *Proc. IEEE 11th Uttar Pradesh Sect. Int. Conf. Electr., Electron. Comput. Eng. (UPCON)*, Lucknow, India, Nov. 2024, pp. 1–7, doi: 10.1109/UPCON62832.2024.10982923.

[24] C. Pornprasit and C. K. Tantithamthavorn, "DeepLineDP: Towards a deep learning approach for line-level defect prediction," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, pp. 84–98, Jan. 2023, doi: 10.1109/TSE.2022.3144348.

[25] T. Zimmermann, N. Nagappan, H. Gall, E. Giger, and B. Murphy, "Cross-project defect prediction: A large scale experiment on data vs. domain vs. process," in *Proc. 7th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng.*, Aug. 2009, pp. 91–100, doi: 10.1145/1595696.1595713.

[26] W. Zheng, L. Tan, and C. Liu, "Software defect prediction method based on transformer model," in *Proc. IEEE Int. Conf. Artif. Intell. Comput. Appl. (ICAICA)*, Jun. 2021, pp. 670–674, doi: 10.1109/ICAICA52286.2021.9498179.

[27] A. Amin, L. Grunske, and A. Colman, "An approach to software reliability prediction based on time series modeling," *J. Syst. Softw.*, vol. 86, no. 7, pp. 1923–1932, Jul. 2013, doi: 10.1016/j.jss.2013.03.045.

[28] G. K. Rajbahadur, S. Wang, G. A. Oliva, Y. Kamei, and A. E. Hassan, "The impact of feature importance methods on the interpretation of defect classifiers," *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2245–2261, Jul. 2022, doi: 10.1109/TSE.2021.3056941.

[29] H. Alyami, M. Nadeem, W. Alosaimi, A. Alharbi, R. Kumar4, B. K. Gupta, A. Agrawal, and R. A. Khan, "Analyzing the data of software security lifespan: Quantum computing era," *Intell. Automat. Soft Comput.*, vol. 31, no. 2, pp. 707–716, 2022, doi: 10.32604/iasc.2022.020780.

[30] M. C. Devi and T. D. Rajkumar, "A novel attention based deep learning model for software defect prediction with bidirectional word embedding system," *Soft Comput.*, vol. 29, pp. 2171–2188, Mar. 2025, doi: 10.1007/s00500-025-10475-5.

[31] F. Rahman and P. Devanbu, "How, and why, process metrics are better," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, May 2013, pp. 432–441, doi: 10.1109/ICSE.2013.6606589.

[32] J. Cito, I. Dillig, V. Murali, and S. Chandra, "Counterfactual explanations for models of code," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 125–134, doi: 10.1145/3510457.3513081.

[33] Y. Hu, W. Luo, and Z. Hu, "A practical approach to explaining defect proneness of code commits by causal discovery," *Eng. Appl. Artif. Intell.*, vol. 123, Feb. 2023, Art. no. 106187, doi: 10.1016/j.engappai.2023.106187.

[34] J. Campos and N. Shakhovska, "Advancing XAI development: An agile framework for human-centered and explainable AI," in *Proc. Int. Conf. Hum.-Comput. Interact.*, 2025, pp. 22–40, doi: 10.1007/978-3-031-93412-4_2.

[35] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "The impact of automated parameter optimization on defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 683–711, Jul. 2019, doi: 10.1109/TSE.2018.2794977.

[36] N. A. Sharma, R. R. Chand, Z. Buksh, A. B. M. S. Ali, A. Hanif, and A. Beheshti, "Explainable AI frameworks: Navigating the present challenges and unveiling innovative applications," *Algorithms*, vol. 17, no. 6, p. 227, 2024, doi: 10.3390/a17060227.

[37] S. Kirmani and M. Shankar, "Generating keywords by associative context with input words," U.S. Patent 10,699,302 B2, Jun. 30, 2020.

[38] T. Abbas, S. A. Rathore, A. Turki, S. Khan, O. Alghushairy, and A. Daud, "Enhancing software engineering with AI: Innovations, challenges, and future directions," *IET Softw.*, vol. 2025, Jun. 2025, Art. no. 5691460, doi: 10.1049/sfw2/5691460.

[39] M. Shepperd, D. Bowes, and T. Hall, "Researcher bias: The use of machine learning in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 40, no. 6, pp. 603–616, Jun. 2014, doi: 10.1109/TSE.2014.2322358.

[40] M. White, C. Vendome, M. Linares-Vasquez, and D. Poshyvanyk, "Toward deep learning software repositories," in *Proc. IEEE/ACM 12th Work. Conf. Mining Softw. Repositories*, May 2015, pp. 334–345, doi: 10.1109/MSR.2015.38.

[41] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[42] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, Nov. 2011.
[43] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," *Intell. Data Anal.*, vol. 6, no. 5, pp. 429–449, 2002.
[44] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," Tech. Rep., 2015.
[45] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," 2017, *arXiv:1702.08608*.
[46] Z. C. Lipton, "The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery," *Queue*, vol. 16, no. 3, pp. 31–57, Jun. 2018, doi: 10.1145/3236386.3241340.
[47] I. Guyon, S. Gunn, M. Nikravesh, and L. A. Zadeh, *Feature Extraction: Foundations and Applications*. Cham, Switzerland: Springer, 2006.
[48] W. McKinney, "Data structures for statistical computing in Python," in *Proc. Python Sci. Conf.*, 2010, pp. 56–61.
[49] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 2623–2631.
[50] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–11.
[51] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 24, 2011, pp. 1–8.
[52] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
[53] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surveys*, vol. 51, no. 5, pp. 1–42, Sep. 2019.
[54] S. Wiegreffe and Y. Pinter, "Attention is not not explanation," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 11–20.
[55] S. Jain and B. C. Wallace, "Attention is not explanation," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol. (NAACL-HLT)*, Jun. 2019, pp. 3543–3556, doi: 10.18653/v1/n19-1357.

ABDULAZIZ ATTAALLAH received the Doctor of Philosophy (Ph.D.) degree in computer software engineering from North Dakota State University. He is currently an Assistant Professor, with a demonstrated history of working in the higher education industry. His skilled in PHP, Java, Software Development, ASP.NET, and C#.

KHALIL AL SULBI received the Doctor of Philosophy (Ph.D.) degree in computer software engineering. He is currently an Assistant Professor with Umm Al-Qura University, with a demonstrated history of working in the higher education industry. His research interests include artificial neural networks, artificial intelligence, and big data.